

Informe de Implementación — Shooter 3D Doom

Nombre: Mihai Flaviu Vescan

Registro: 218058349

Video: <https://www.mediafire.com/file/r8eewdt81ifyp8q/Video.mp4>

1. Sistema de munición y recarga

Se extendió el script existente *Disparar.cs* con un cargador limitado ($\text{balasMax} = 3$) y una reserva finita (balasReserva), ambos configurables desde el Inspector. El disparo se bloquea si el cargador está vacío o hay una recarga en curso. La tecla R inicia la recarga solo si falta munición y hay reserva; la espera de 1,5 s se implementó con `Invoke()`, la solución más simple para un retardo único. Al terminar, se transfieren balas desde la reserva con `Mathf.Min(faltantes, reserva)`, evitando crear munición de la nada. El HUD (*MunicionHUD.cs*) lee el estado mediante métodos públicos de solo lectura (BalasActuales , Recargando), manteniendo el estado encapsulado: el arma es la única dueña de sus datos y la UI solo consulta. Se muestran los estados "Balas: x/3 Reserva: n", "RECARGANDO..." y "SIN MUNICIÓN".

2. Agregar enemigos

Cada enemigo es un prefab con *NavMeshAgent*, *Capsule Collider*, el componente compartido *Vida* y el script *Enemigo.cs*. La navegación usa un *NavMeshSurface* horneado sobre la geometría del laberinto: el agente calcula solo las rutas y esquiva paredes con `SetDestination(jugador.position)`. El jugador se localiza por tag ("Player"), desacoplando el script de referencias manuales. La conducta se decide por distancia: lejos persigue; dentro del rango de ataque (8 m) se detiene y dispara cada 1,5 s. El ataque replica la mecánica del jugador: un *Raycast* que aplica `RecibirDano()` al impactar. El rayo además garantiza línea de visión, impidiendo disparos a través de paredes. El aspecto estilo Doom se logró con un sprite hijo y un script billboard (*Camara.cs*) que lo orienta siempre hacia la cámara.

3. Contador de enemigos y condición doble de victoria

Ambas funciones comparten una misma fuente de verdad: `FindObjectsByType<Enemigo>()` cuenta los enemigos vivos en la escena. Como *Vida* destruye al enemigo al morir, el conteo se actualiza solo, sin contadores manuales que puedan desincronizarse. El HUD (*ContadorEnemigos.cs*) muestra ese número, y la zona de llegada (*Meta.cs*, un collider trigger) solo dispara la victoria si el jugador entra (verificado por tag) y el conteo es cero; en caso contrario informa por consola cuántos faltan. La coherencia entre HUD y condición de victoria queda garantizada por usar la misma consulta.

4. Menú de Game Over con reinicio

Se reemplazó la recarga inmediata de escena por un flujo de menú: al morir el jugador, *Vida.cs* delega en *MenuGameOver.cs*, que activa un panel de UI, pausa el juego con `Time.timeScale = 0` (congela movimiento, disparos e `Invokes` de los enemigos) y libera el cursor (`CursorLockMode.None`) para poder hacer clic. El botón "Reintentar" restaura `timeScale = 1` antes de recargar la escena — orden crítico, pues la escena nueva heredaría el tiempo congelado. El mismo componente gestiona también el panel de Victoria, reutilizando la pausa y el botón de reinicio.

5. Animación o feedback de dano

Se usó una *Image* de UI roja a pantalla completa con alfa inicial 0 (invisible) y *Raycast Target* desactivado para no bloquear los botones. Al recibir daño, *Vida.cs* invoca `DanoPantalla.Parpadear()`, que fija el alfa en 0,5; el `Update()` del efecto lo desvanece gradualmente con `Time.deltaTime`, logrando un flash suave e independiente del framerate. El efecto se dispara antes de evaluar la muerte, de modo que el golpe letal también parpadea. Se complementó con sonidos de daño diferenciados (jugador_dano / enemigo_dano) reproducidos con `PlayClipAtPoint`, que sobrevive a la destrucción del objeto emisor.